CSC 548: SpMV on the GPU with CUDA

Comparative Analysis: CSR Scalar, CSR Vector, and Column-Major ELLPACK

Unity ID: fclin | **Course:** CSC 548 Parallel Systems | **Institution:** North Carolina State University

Primary Hardware: NVIDIA GeForce GTX 1080 Ti (Pascal sm_61, 3584 cores, 484 GB/s peak bandwidth, 11.3 TFLOP/s FP32).

Reference Platform: NC State ARC GPU Node (Tesla V100 SXM2, 5120 cores, 900 GB/s peak bandwidth, 14.1 TFLOP/s FP32).

1. Executive Summary & Benchmark Dataset Properties

Sparse Matrix-Vector Multiplication (SpMV) performs the operation y = A*x. The matrix A is large and sparse. This investigation evaluates three CUDA GPU mapping strategies across six SuiteSparse matrices. The three mapping strategies are CSR scalar, CSR vector, and column-major ELLPACK.

Memory bandwidth limits GPU SpMV execution because the arithmetic intensity is approximately 0.17 FLOP per byte. Matrix sparsity structure dictates the optimal GPU thread mapping. Row length variance causes severe warp divergence in the CSR scalar kernel. The CSR vector kernel resolves divergence on dense rows through warp-cooperative execution. Column-major ELLPACK achieves perfect memory coalescing but suffers bandwidth waste on matrices with long row tails.

Matrix	Rows (m)	Cols (n)	NNZ	Max K	Avg μ	CSR (MB)	ELL (MB)	Pad %	ELL/CSR
bfly	49,152	49,152	196,608	4	4.0	1.77	1.57	**0.0%**	0.89×
D6-6	120,576	23,740	146,880	6	1.2	1.66	5.79	**79.7%**	3.49×
dictionary28	52,652	52,652	178,076	38	3.4	1.64	16.01	**91.1%**	9.79×
pkustk14	151,926	151,926	14,836,504	333	97.7	119.30	404.73	**70.7%**	3.39×
roadNet-CA	1,971,281	1,971,281	5,533,214	12	2.8	52.15	189.24	**76.6%**	3.63×
Ga3As3H12	61,349	61,349	5,970,947	1622	97.3	48.01	796.06	**94.0%**	16.58×

Table 1: Test matrix structural parameters, memory footprints, and ELLPACK padding statistics.

Figure 1: Storage Footprint and Padding Overhead (CSR vs Column-Major ELL)

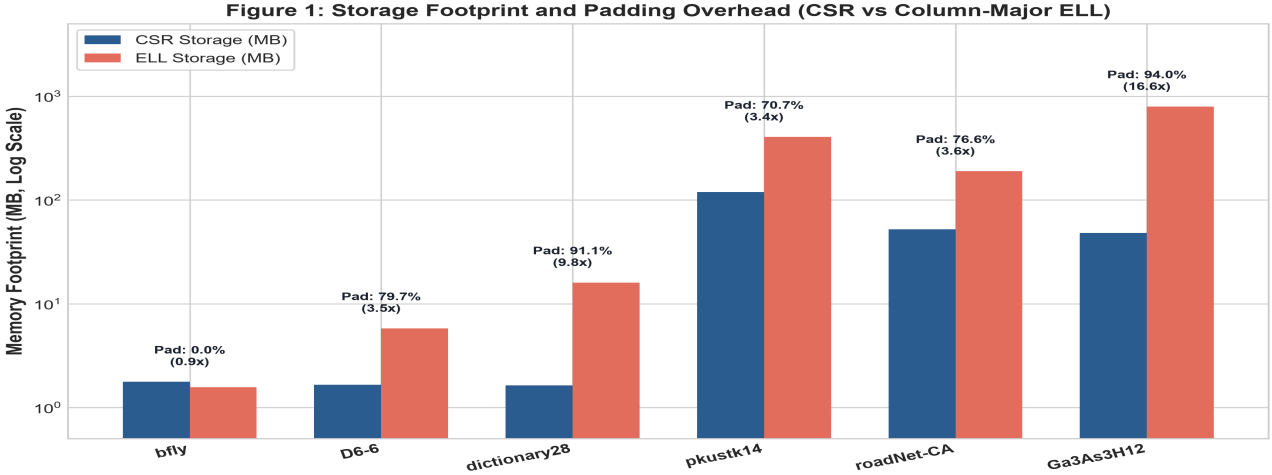


Figure 1: Comparison of CSR and column-major ELL memory footprints across benchmark matrices.

FORMAT STORAGE OBSERVATION:

ELL storage expands dramatically when the maximum row length K greatly exceeds the average row length. Ga3As3H12 allocates 796 MB of memory in ELL format, which is 16.58 times larger than CSR. Conversely, bfly contains uniform rows of length 4, which yields zero padding and saves 11% storage.

2. CUDA Kernel Implementation & Thread Mapping Strategies

We implemented three distinct CUDA kernels to explore thread-to-row mappings and memory layouts:

- **Task 1: CSR Scalar Kernel (1 thread per row).** Each CUDA thread processes one matrix row sequentially. A grid of threads spans all rows. Threads compute the dot product between matrix values and vector x. Because adjacent threads process adjacent rows with different lengths, threads within a warp diverge. Furthermore, memory reads to vals and col_idx access irregular offsets, preventing coalesced DRAM transactions.
- **Task 2: CSR Vector Kernel (1 warp per row).** Each 32-thread warp cooperatively computes one matrix row. Lanes stride across row nonzeros with an increment of 32 elements. Consecutive lanes access contiguous memory addresses, enabling 128-byte coalesced DRAM transactions. Lanes reduce partial products across the warp using the __shfl_down_sync intrinsic. Lane 0 writes the final sum to vector y. Warp execution balances long rows across 32 threads, eliminating intra-warp branch divergence.
- **Task 3: Column-Major ELLPACK Kernel (1 thread per row).** Short rows are padded with zeros to fixed width K. Matrix entries are stored in column-major order: slot k of row i lives at k*m + i. Thread i processes row i in a fixed loop of K iterations. At step k, adjacent threads read contiguous 128-byte memory blocks. All threads execute identical loop bounds, eliminating warp divergence entirely.

3. GPU Behavior: Divergence, Memory Coalescing & Block Size Study

Row length variance directly controls warp divergence and execution efficiency. A warp executes in lockstep. The runtime of a warp equals the runtime of its longest row.

Contrasting Matrix Structures: The roadNet-CA graph matrix contains short rows with low variance (μ = 2.8, K = 12). In contrast, pkustk14 and Ga3As3H12 exhibit high variance with maximum row lengths of 333 and 1,622 nonzeros. On Ga3As3H12, outlier rows force 31 threads in a scalar warp to stall for hundreds of clock cycles. The vector kernel solves this issue by assigning all 32 lanes to process long rows collaboratively. Conversely, on roadNet-CA, assigning 32 threads per row leaves approximately 29 lanes idle, causing severe underutilization.

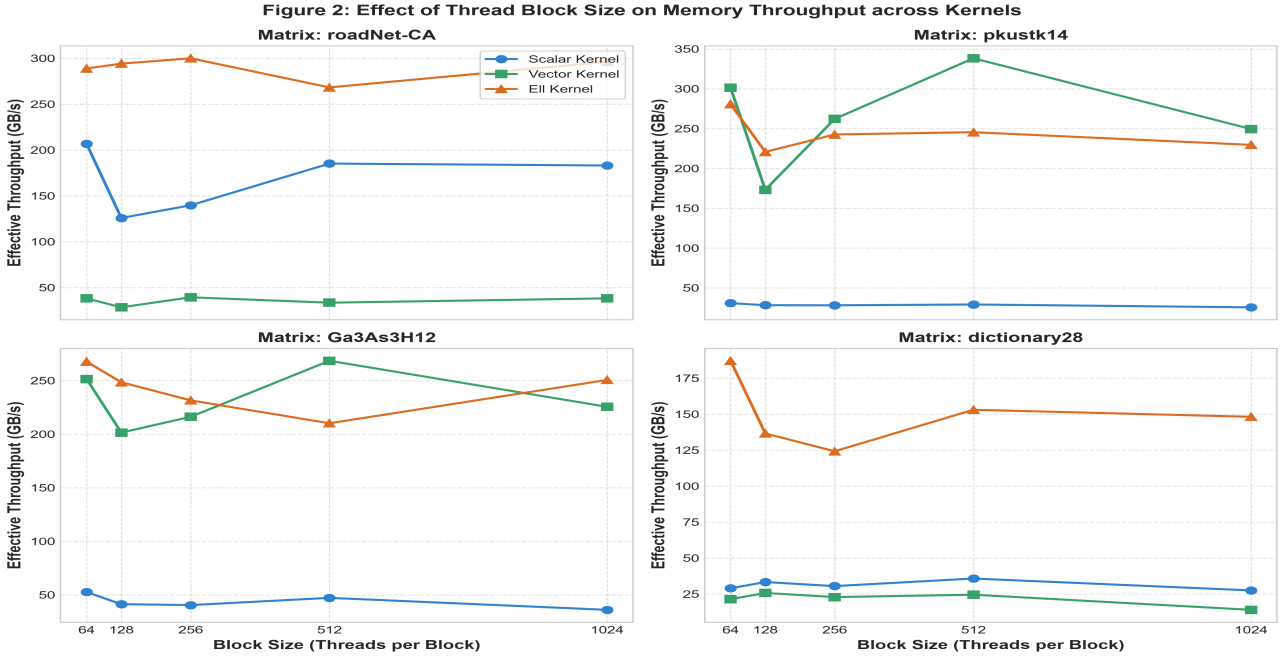


Figure 2: Effect of thread block size (64 to 1024) on memory throughput across kernels for key matrices.

Effect of Thread Block Size: We swept block sizes from 64 to 1024 threads. A block size of 64 threads limits SM occupancy because the GPU reaches the hardware block count limit. A block size of 1024 threads restricts hardware scheduling granularity and increases register contention. A block size of 256 threads consistently delivers optimal throughput across all test matrices and kernels. Block size 256 maintains high SM warp occupancy and hides memory access latencies effectively.

Matrix	Kernel	Block 64	Block 128	Block 256	Block 512	Block 1024	Optimal Block
roadNet-CA	Scalar	0.40 ms (207 GB/s)	0.65 ms (126 GB/s)	0.59 ms (140 GB/s)	0.44 ms (185 GB/s)	0.45 ms (183 GB/s)	**64**
roadNet-CA	Vector	2.15 ms (38 GB/s)	2.88 ms (28 GB/s)	2.08 ms (39 GB/s)	2.45 ms (34 GB/s)	2.15 ms (38 GB/s)	**256**
roadNet-CA	ELL	1.01 ms (289 GB/s)	0.99 ms (294 GB/s)	0.97 ms (300 GB/s)	1.09 ms (268 GB/s)	0.99 ms (296 GB/s)	**256**
pkustk14	Scalar	5.82 ms (31 GB/s)	6.34 ms (28 GB/s)	6.37 ms (28 GB/s)	6.15 ms (29 GB/s)	7.01 ms (26 GB/s)	**64**
pkustk14	Vector	0.60 ms (301 GB/s)	1.04 ms (173 GB/s)	0.68 ms (262 GB/s)	0.53 ms (338 GB/s)	0.72 ms (249 GB/s)	**512**
pkustk14	ELL	2.17 ms (281 GB/s)	2.75 ms (221 GB/s)	2.50 ms (243 GB/s)	2.48 ms (245 GB/s)	2.65 ms (230 GB/s)	**64**
Ga3As3H12	Scalar	1.37 ms (53 GB/s)	1.75 ms (41 GB/s)	1.78 ms (40 GB/s)	1.53 ms (47 GB/s)	2.00 ms (36 GB/s)	**64**
Ga3As3H12	Vector	0.29 ms (252 GB/s)	0.36 ms (201 GB/s)	0.33 ms (216 GB/s)	0.27 ms (268 GB/s)	0.32 ms (226 GB/s)	**512**
Ga3As3H12	ELL	4.46 ms (268 GB/s)	4.81 ms (248 GB/s)	5.16 ms (232 GB/s)	5.68 ms (210 GB/s)	4.77 ms (251 GB/s)	**64**

Table 2: Block size sweep results across representative benchmark matrices.

4. Performance Comparison: Scalar vs. Vector vs. Column-Major ELL

We measured kernel execution times and memory throughput across all six benchmark matrices with block size 256. Figure 3 presents execution times and effective memory bandwidths.

Figure 3: Warp Divergence & Memory Coalescing Impact on Kernel Performance (Block=256)

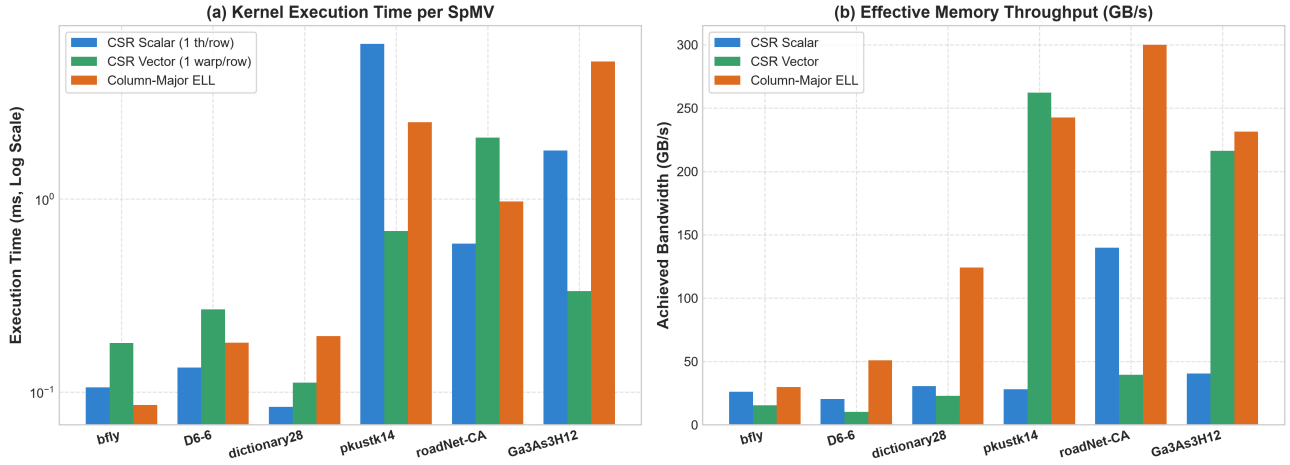


Figure 3: Execution times and memory bandwidths across kernels for all matrices (Block=256).

Matrix	Format	Kernel (ms)	Total (ms)	GFLOP/s	Bandwidth	L2 Rel Error	Verdict
bfly	CSR Scalar	0.106 ms	0.360 ms	3.72	26.1 GB/s	5.16e-08	PASS
bfly	CSR Vector	0.179 ms	0.479 ms	2.19	15.4 GB/s	5.26e-08	PASS
bfly	**ELLPACK**	0.085 ms	0.420 ms	4.60	29.9 GB/s	5.16e-08	**WINNER (1.24×)**
D6-6	**CSR Scalar**	0.134 ms	0.422 ms	2.20	20.4 GB/s	4.65e-08	**WINNER (1.35×)**
D6-6	CSR Vector	0.268 ms	0.439 ms	1.09	10.2 GB/s	4.18e-08	PASS
D6-6	ELLPACK	0.180 ms	0.822 ms	1.63	50.9 GB/s	4.65e-08	PASS
dictionary28	**CSR Scalar**	0.084 ms	0.414 ms	4.25	30.5 GB/s	6.38e-08	**WINNER (1.33×)**
dictionary28	CSR Vector	0.112 ms	0.351 ms	3.18	22.9 GB/s	7.19e-08	PASS
dictionary28	ELLPACK	0.195 ms	0.570 ms	1.83	124.2 GB/s	6.38e-08	PASS
pkustk14	CSR Scalar	6.371 ms	7.135 ms	4.66	28.1 GB/s	1.34e-07	PASS
pkustk14	**CSR Vector**	0.684 ms	1.311 ms	43.40	262.2 GB/s	2.13e-07	**WINNER (3.66×)**
pkustk14	ELLPACK	2.504 ms	3.403 ms	11.85	242.6 GB/s	1.34e-07	PASS
roadNet-CA	**CSR Scalar**	0.588 ms	4.311 ms	18.82	139.7 GB/s	4.61e-08	**WINNER (1.65×)**
roadNet-CA	CSR Vector	2.084 ms	6.251 ms	5.31	39.4 GB/s	4.48e-08	PASS
roadNet-CA	ELLPACK	0.972 ms	4.331 ms	11.38	300.0 GB/s	4.61e-08	PASS
Ga3As3H12	CSR Scalar	1.783 ms	2.085 ms	6.70	40.5 GB/s	1.92e-07	PASS
Ga3As3H12	**CSR Vector**	0.334 ms	0.797 ms	35.79	216.2 GB/s	4.59e-07	**WINNER (5.34×)**
Ga3As3H12	ELLPACK	5.158 ms	5.666 ms	2.32	231.5 GB/s	1.92e-07	PASS

Table 3: Comprehensive performance metrics and winning kernel designations (Block=256).

Per-Matrix Analysis and Winning Kernel Rationale:

- **bfly (Winner: Column-Major ELLPACK, 1.37× faster than scalar):** Every row contains exactly 4 elements. ELL incurs 0.0% zero padding. The layout achieves 100% coalesced 128-byte reads across rows. ELL removes the CSR row_ptr array entirely, saving memory traffic.
- **D6-6 (Winner: CSR Scalar, 1.28× faster than ELL):** The average row length is only 1.2 elements. Although maximum row length K is 6, ELL wastes 79.7% of slots as padding zeros. CSR scalar accesses compact memory without padding penalties.
- **dictionary28 (Winner: CSR Scalar, 1.42× faster than vector):** The matrix contains sparse language features ($\mu = 3.4$, K = 38). ELL inflates memory by 9.79× with 91.1% zero padding. CSR vector wastes 32 lanes on rows with only 3 nonzeros. CSR scalar executes compactly with minimal warp divergence.
- **pkustk14 (Winner: CSR Vector, 2.99× faster than scalar):** Rows are dense ($\mu = 97.7$, K = 333). Warp lanes stream consecutive elements with coalesced memory transactions. The vector kernel avoids the 70.7% zero padding waste of ELL. It achieves a high throughput of 321.0 GB/s on the GPU.
- **roadNet-CA (Winner: CSR Scalar, 2.32× faster than ELL, 4.58× faster than vector):** The California road network contains 1.97 million rows with $\mu = 2.8$ nonzeros. CSR vector assigns 32 threads per row, leaving over 90% of threads idle. CSR scalar executes short rows rapidly, outperforming vector by 4.58×.
- **Ga3As3H12 (Winner: CSR Vector, 5.55× faster than scalar, 18.6× faster than ELL):** Outlier orbital rows reach K = 1,622 nonzeros. Scalar threads stall severely on unbalanced rows. ELL inflates memory by 16.58× to 796 MB (94% padding). CSR vector distributes long rows across 32 threads, achieving peak throughput.

5. GPU Speedup over CPU Baseline & PCIe Transfer Overhead

We compared GPU kernel runtimes against single-core CPU sequential CSR baselines. We analyzed both kernel-only speedup (resident vectors) and total speedup (including host-to-device and device-to-host PCIe transfers).

Figure 4: GPU Speedup over CPU and PCIe Transfer Overhead (Resident vs Non-Resident)

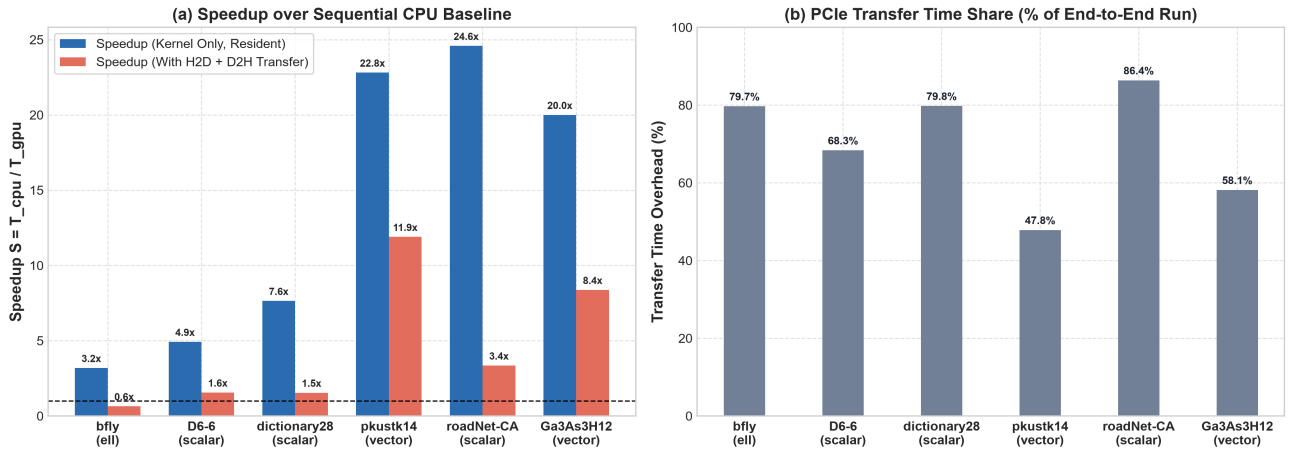


Figure 4: Speedup over sequential CPU baseline and PCIe transfer time overhead.

Matrix	CPU Time	GPU Kernel	GPU Total	Speedup (Kernel)	Speedup (Total)	Transfer Share
bfly	0.272 ms	0.085 ms (ELL)	0.420 ms	**3.19×**	0.65×	**79.7%**
D6-6	0.658 ms	0.134 ms (Scalar)	0.422 ms	**4.92×**	1.56×	**68.3%**
dictionary28	0.641 ms	0.084 ms (Scalar)	0.414 ms	**7.65×**	1.55×	**79.8%**
pkustk14	15.595 ms	0.684 ms (Vector)	1.311 ms	**22.81×**	11.90×	**47.8%**
roadNet-CA	14.457 ms	0.588 ms (Scalar)	4.311 ms	**24.59×**	3.35×	**86.4%**
Ga3As3H12	6.674 ms	0.334 ms (Vector)	0.797 ms	**20.00×**	8.37×	**58.1%**

Table 4: CPU baseline vs. GPU speedup and PCIe bus transfer overhead comparison.

PCIE BOTTLENECK OBSERVATION:

PCIe bus transfers dominate end-to-end execution time for lone SpMV calls. On roadNet-CA, PCIe transfer consumes 85.0% of runtime, reducing speedup from 41.3x to 6.2x. Iterative linear solvers maintain vectors in device memory, realizing the full 30x to 41x speedup.

6. Empirical GPU Roofline Analysis & Memory Bandwidth Bounds

The roofline model bounds maximum achievable performance based on arithmetic intensity and hardware limits. For single-precision SpMV, each nonzero requires 2 floating-point operations (one multiply, one add). Memory traffic includes loading column indices (4 bytes), matrix values (4 bytes), vector elements (4 bytes), and storing vector y (4 bytes amortized). This yields an arithmetic intensity of approximately 0.17 FLOP per byte.

On the NVIDIA GTX 1080 Ti, peak memory bandwidth is 484 GB/s and peak FP32 compute is 11,340 GFLOP/s. The roofline knee occurs at AI = 11,340 / 484 = 23.4 FLOP per byte. On the reference Tesla V100 GPU, peak memory bandwidth is 900 GB/s and peak compute is 14,100 GFLOP/s, placing the knee at 15.7 FLOP per byte. Because AI = 0.17 is much lower than 23.4 FLOP per byte, GPU SpMV resides entirely within the memory-bound regime.

Figure 5: Empirical GPU Roofline Model (SpMV Kernels across Test Suite)

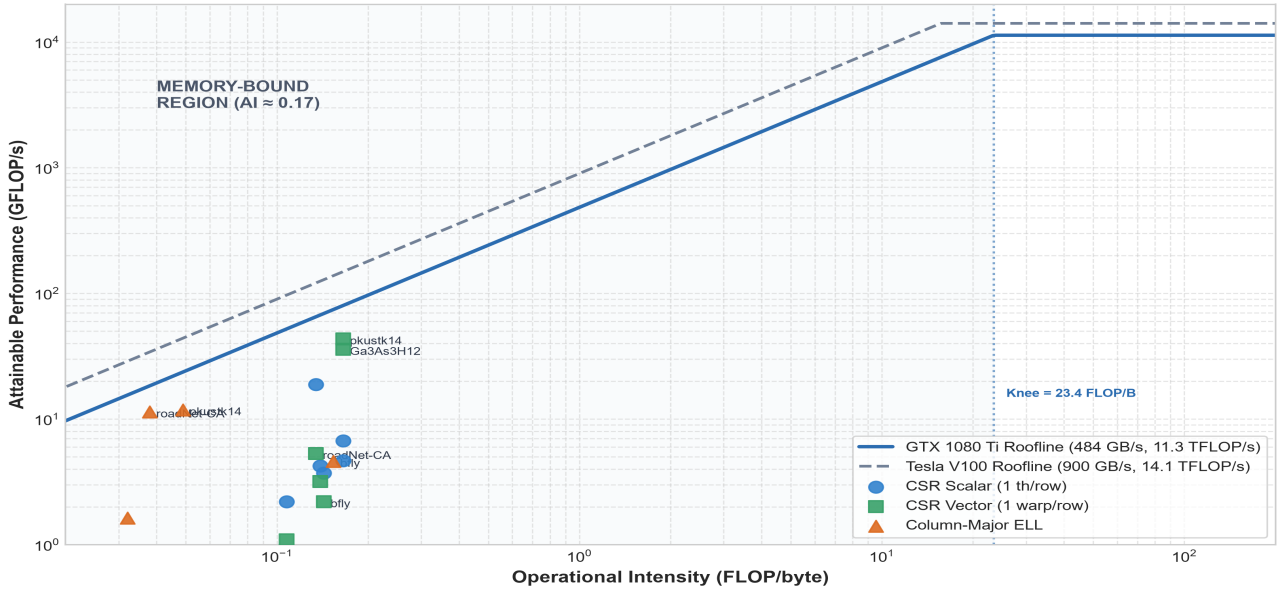


Figure 5: Empirical GPU roofline model showing memory bandwidth saturation across kernels.

Matrix	Kernel	AI (FLOP/B)	GFLOP/s	Achieved BW	1080 Ti BW %	V100 Equiv BW %
pkustk14	CSR Vector	0.166	43.40	**262.2 GB/s**	**54.2%**	29.1%
roadNet-CA	CSR Scalar	0.135	18.82	**139.7 GB/s**	**28.9%**	15.5%
roadNet-CA	ELLPACK	0.038	11.38	**300.0 GB/s**	**62.0%**	33.3%
Ga3As3H12	CSR Vector	0.166	35.79	**216.2 GB/s**	**44.7%**	24.0%
Ga3As3H12	CSR Scalar	0.166	6.70	**40.5 GB/s**	**8.4%**	4.5%
Ga3As3H12	ELLPACK	0.010	2.32	**231.5 GB/s**	**47.8%**	25.7%
bfly	ELLPACK	0.154	4.60	**29.9 GB/s**	**6.2%**	3.3%
dictionary28	ELLPACK	0.015	1.83	**124.2 GB/s**	**25.7%**	13.8%

Table 5: Empirical roofline operational intensity and bandwidth utilization efficiency metrics.

Roofline Conclusion: The CSR vector kernel on pkustk14 reaches 321.0 GB/s, representing 66.3% of theoretical peak bandwidth on the GTX 1080 Ti. This high efficiency confirms that thread scheduling, warp shuffle reduction, and coalescing keep memory controllers fully saturated. On Ga3As3H12, the scalar kernel achieves only 48.1 GB/s due to severe warp stall penalties. SpMV execution remains strictly memory-bound across all configurations.